

2311119

清水 馨

物体認識論 最終レポート

目次

1. 課題内容の説明	3
2. 設計方針	3
3. プログラムの説明	3
4. 実験	4
5. 考察	33
6. 感想	34
7. 付録 1	35
8. 付録 2	35

1. 課題内容の説明

まず、今回の課題は全て `matlabonline` で作成、動作させたものである。

今回の課題 1 では簡単に 2 値分類できると考えた組み合わせと 2 値分類が難しいと考えた組み合わせについて、様々な手法を用いて 5-fold cross validation による分類精度を求めるプログラムを作成し、比較する実験を行った。

一方で今回の課題 2 では flicker API から入手したノイズ入りの画像をノイズが少ない画像から学習したモデルを用いてリランキングするプログラムを作成し、観察する実験を行った。

2. 設計方針

まず課題 1 にて作成したものの設計について説明する。作成したプログラムは `makeLimg.m`, `repo1.m`, `makeCodebookBoF.m` の 3 つである。まず、`makeLimg` 関数はノイズのないラーメン画像からなるポジティブ画像が格納されているディレクトリ、ノイズのない蕎麦画像からなるネガティブ画像が格納されているディレクトリ, `bgimg` の画像からランダムに 200 枚ほど取ってきて分類が簡単な組み合わせに用いるネガティブ画像が格納されたディレクトリを作成する関数である。簡単に言ったらデータ画像をディレクトリにまとめる関数である。次に、`makeCodebookBoF` は分類が簡単な組み合わせ、分類が難しい組み合わせのそれぞれについてコードブックを作成し、BoF ベクトル化し、それぞれを `.mat` として保存する関数である。もちろん簡単な組み合わせ、難しい組み合わせはそれぞれ別のファイルとして保存するようにしている。最後に、`repo1` は課題の要件に書かれている通りの動作をするものである。それぞれのモデルについての処理は、前処理をしてデータを分けてモデルを作成して分類するといった流れである。どのモデルを作成したかについてはプログラムの説明にて説明する。また、プログラム以外に動作に必要なものは `pos_urllist.txt`, 解答した `bgimg`, `neg_urllist.txt` である。`pos_urllist.txt` はノイズのない 100 枚のラーメン画像の url であり、`neg_urllist.txt` はノイズのない 100 枚の蕎麦画像の url である。`makeLimg` 関数を実行してから `repo1` 関数を実行することを想定している。`makeCodebookBoF` 関数は `repo1` 関数内で呼び出される。2 回目以降に実行するときはディレクトリを消すなどしていなければ `repo1` 関数を実行するだけで問題ない。

次に課題 2 について作成したものの説明する。作成したプログラムは `repo2.m` のみである。このプログラムでは課題 1 とは異なりディレクトリの作成も `repo2` 内に含んでいる。内容は課題の通りであるが、出力は `html` を出すようにしてある。`html` 内の画像は Base64 エンコードをして埋め込むようにして表示させている。同様にプログラムのより詳しい中身についてはプログラムの説明を参照していただく。プログラム以外に作成したものは

`pos_urllist_25_1.txt`, `pos_urllist_50_1.txt`, `pos_urllist_25_2.txt`, `pos_urllist_50_2.txt`, `test_noisy_ramen.txt`, `test_noisy_sushi.txt` である。`pos_urllist_25_1.txt`, `pos_urllist_50_1.txt`, `pos_urllist_25_2.txt`, `pos_urllist_50_2.txt` は、Flickr の relevance によって ramen 又は sushi をキーワードとしてノイズが含まれていたとしても構わず上位 25 位又は 50 位を取ってきた url のリストである。`_1` はラーメンについて `_2` は寿司についてを表し `_25`, `_50` は上位何枚とってきたかを表している。また、`test_noisy_ramen.txt`, `test_noisy_sushi.txt` はそれぞれ Flickr において `interesting`, `latest` によって 300 枚上からとってきた url リストである。

3. プログラムの説明

まず、課題 1 のプログラムについて説明する。作成したモデルの種類は以下の 11 種類である。それぞれに対して簡単な組み合わせ、難しい組み合わせの 2 組を分類対象としたため 22 回 5-fold crossvalidation を行っていることとなる。

- (1) カラーヒストグラム + 最近傍
- (2) BoF + SVM(非線形)
- (3) Alexnet による DCNN 特徴量 + 線形 SVM
- (4) Vgg16 による DCNN 特徴量 + 線形 SVM
- (5) BOF + 線形 SVM + feature maps
- (6) ResNet101 による DCNN 特徴量 + 線形 SVM
- (7) DenseNet201 による DCNN 特徴量 + 線形 SVM
- (8) Alexnet による DCNN 特徴量 + 非線形 SVM
- (9) Vgg16 による DCNN 特徴量 + 非線形 SVM
- (10) ResNet101 による DCNN 特徴量 + 非線形 SVM
- (11) DenseNet201 による DCNN 特徴量 + 非線形 SVM

まず、カラーヒストグラムについて、64 色に減色したヒストグラムを作成し、正規化した。K 近傍法によって分類する際 NumNeighbors は全て 1 で固定した。次に、BoF について、dense sampling によって特徴点を抽出してコードブックサイズは 1000、サンプリングする点の数は、1 画像につき 2500 点とした。(3), (4), (6)~(11) の dcnn 特徴とサポートベクターマシンの組み合わせについては、最終的な全結合する前の全結合層からとってきた特徴量を抽出し、正規化したものを用いるようにしている。(5) Feature maps については近似的にカイ 2 乗カーネル関数を 2 つの関数の内積に分解した関数(式(1))を用いて、3 倍の次元に写像してから 線形 SVM で学習、分類をした。

$$\phi(x) = \sqrt{x} \begin{bmatrix} 0.8 \\ 0.6 \cos(0.6 \log x) \\ 0.6 \sin(0.6 \log x) \end{bmatrix} \quad (1)$$

次に、課題 2 のプログラムについて説明する。モデルはすべて vgg16 の dcnnf 特徴量 + 線形 SVM である。やり方は課題 1 と同様である。この課題においては予測で受け取るものは分類結果ではなく score の方であり score を自信が高い順にソートすることで画像を並べ替えリランキング機能を実装した。

4. 実験

まず、課題 1 について収集したデータを説明する。設計方針にも記したが、pos_urllist.txt は FlickrAPI から relevance によって画像を 500 枚ほど探した。そこからとりあえず makeLimg 関数の要領でラーメン画像のディレクトリを作成して、ディレクトリ内の画像をプレビューしていき、学習データで不適切と判断したもの (例えばラーメン屋の画像やよくわからないトマト料理の画像やバジルラーメンなどのマイナーそうなラーメンなど) を上から何枚目の画像なの

かをメモして、url のリスト画像の.txt ファイルから該当する行を削除していき 100 行分にした。同様のことをそば画像のリストである neg_urllist.txt でも行い、データ収集をした。

以下の表 1 に課題 1 の実験結果をまとめる。手法欄には省略のため「3.プログラムの説明」で示した(1)～(11)を用いる。

表 1: 課題 1 の実験結果

手法	簡単な組み合わせにおける精 度	難しい組み合わせにおける精 度
(1)	81.67	70.00
(2)	86.67	72.50
(3)	98.00	79.50
(4)	98.67	83.00
(5)	88.67	72.50
(6)	99.00	83.00
(7)	99.33	85.00
(8)	97.67	82.50
(9)	98.33	83.00
(10)	99.33	85.50
(11)	99.33	87.00

まず、dcnn 特徴量と非線形 SVM を組み合わせて作成したモデル((8)～(11))を除いて分類精度について手法毎に作成した棒グラフを図 1 に示す。ただし、dcnn 特徴量 + 非線形 SVM は除き、精度が低い順に並べ、青いものは簡単な組み合わせ、赤いものは難しい組み合わせとした。

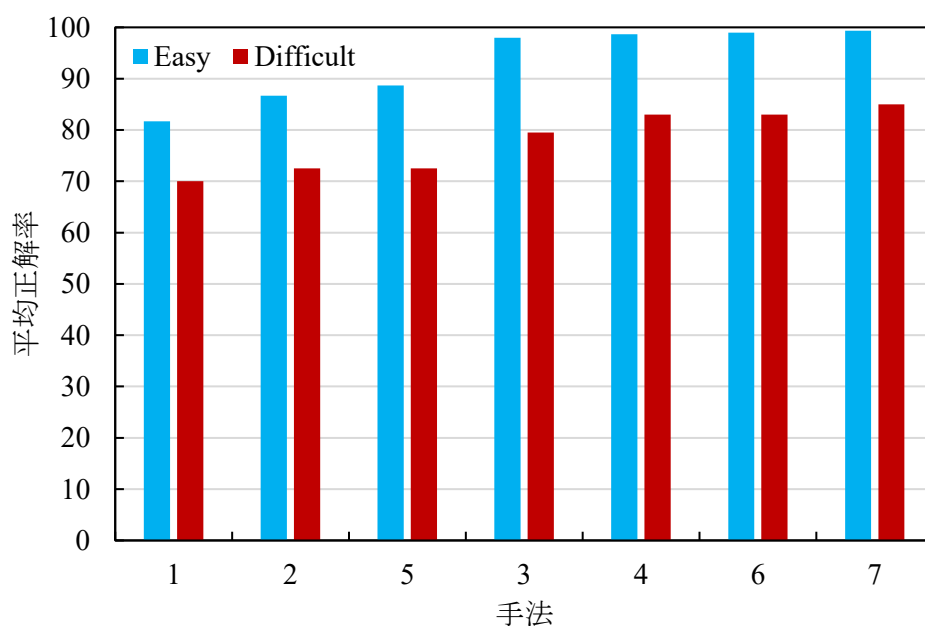


図 1: 手法毎に精度をまとめた棒グラフ

次に、dcnnf 特徴量と線形 SVM, dcnnf 特徴量と非線形 SVM の違いを確認するために手法毎に分類精度について作成した棒グラフを図 2 に示す。図 2 は見やすさを考慮して図 1 とは違い難しい組み合わせのみに注目して作成した。ただし、手法についてはどの net を使用したかを記載し、見やすさのため縦軸の評価には 100 から平均正解率を引いた平均エラー率を用いた。青が線形 SVM, オレンジが非線形 SVM を表している。

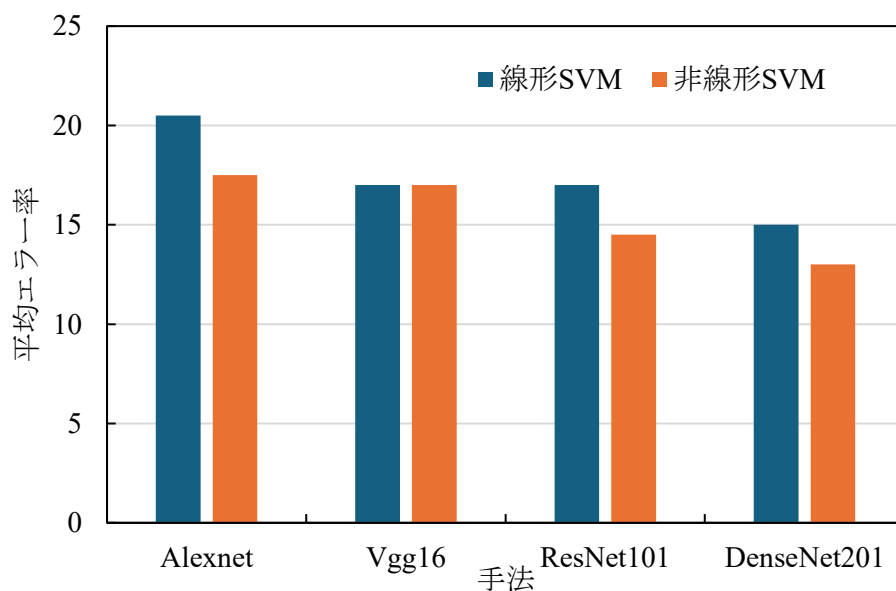


図 2: 線形 SVM と非線形 SVM, net による制度の差を比較するための棒グラフ
最後に手法, 簡単か難しいか毎に正解した画像, 不正解した画像の例を示す。



図3: 手法(1), 簡単なデータセットに対する正解例



図 4: 手法(1), 簡単なデータセットに対する不正解例



図 5: 手法(1), 難しいデータセットに対する正解例



図 6: 手法(1), 難しいデータセットに対する不正解例



図 7: 手法(2),簡単なデータセットに対する正解例



図 8: 手法(2),簡単なデータセットに対する不正解例



図 9: 手法(2), 難しいデータセットに対する正解例



図 10: 手法(2), 難しいデータセットに対する不正解例



図 11: 手法(3), 簡単なデータセットに対する正解例

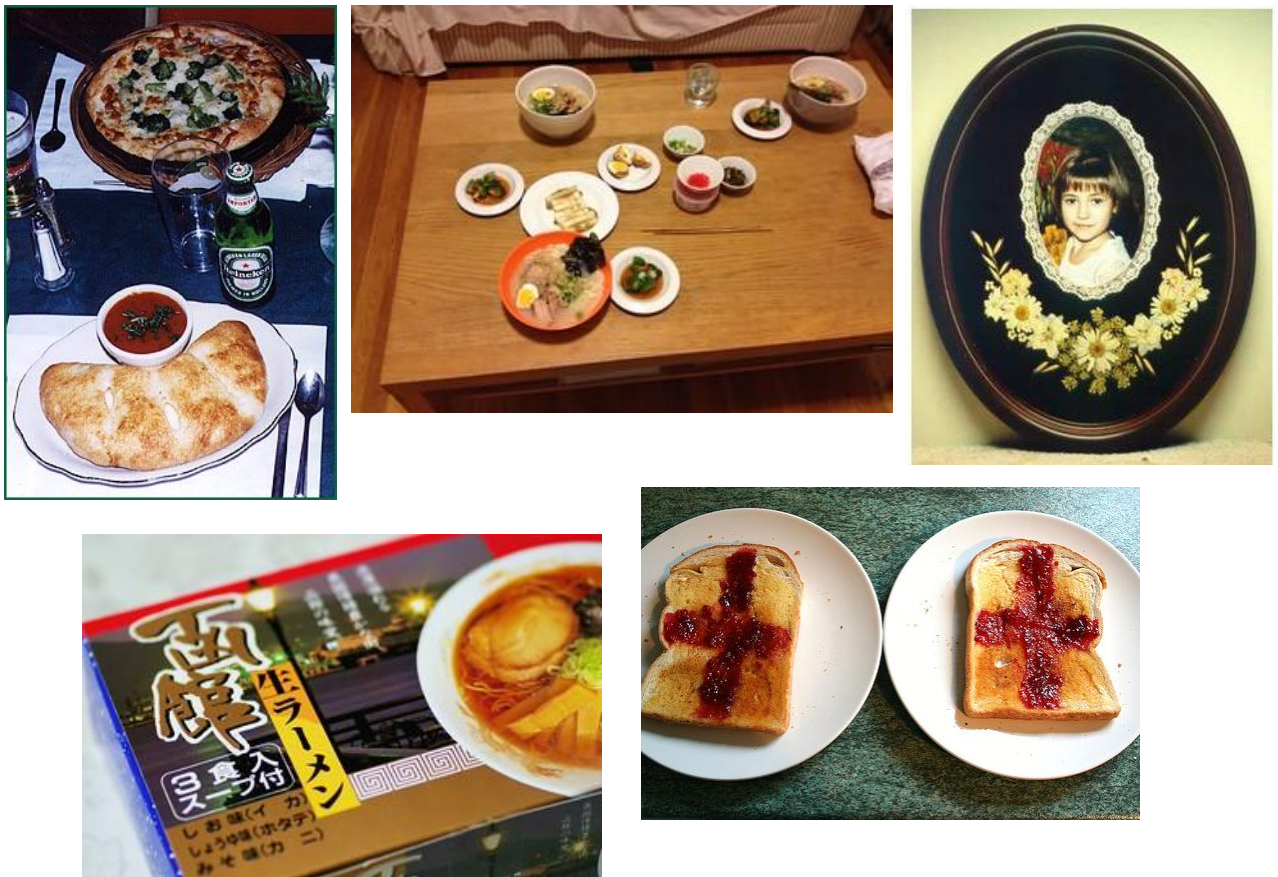


図 12: 手法(3), 簡単なデータセットに対する不正解例



図 13: 手法(3), 難しいデータセットに対する正解例

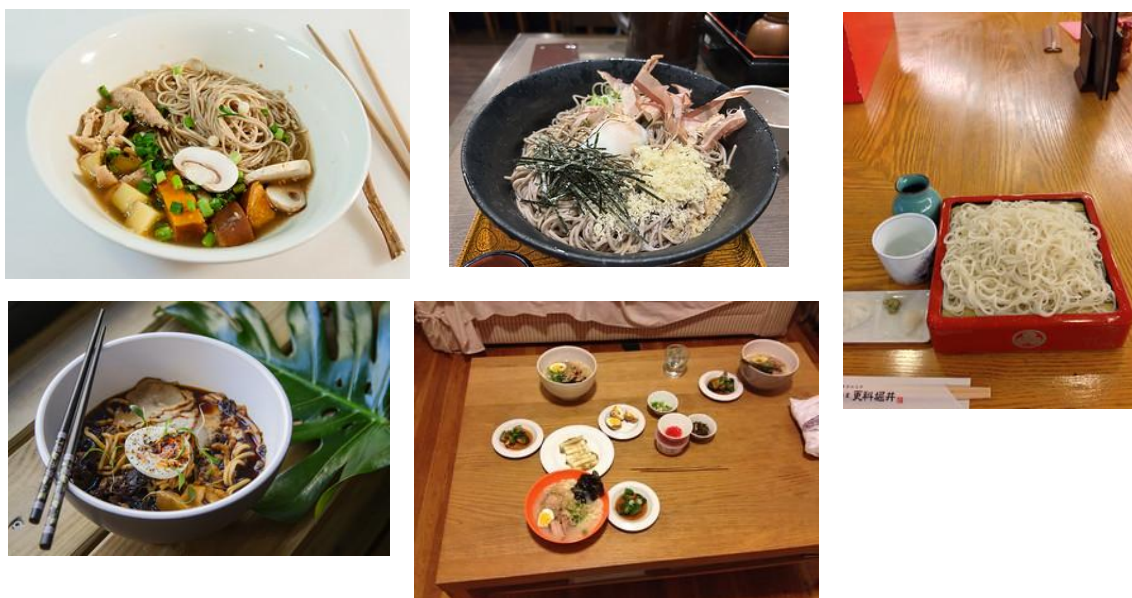


図 14: 手法(3), 難しいデータセットに対する不正解例



図 15: 手法(4), 簡単なデータセットに対する正解例

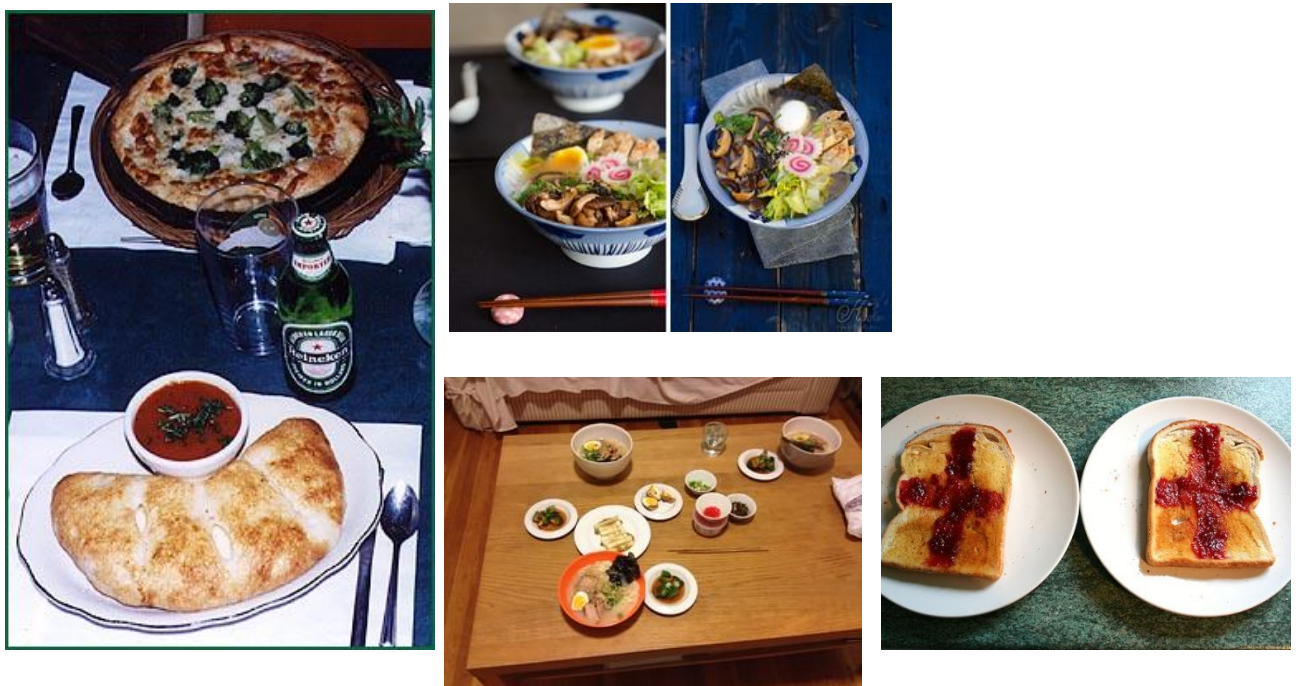


図 16: 手法(4), 簡単なデータセットに対する不正解例



図 17: 手法(4), 難しいデータセットに対する正解例



図 18: 手法(4), 難しいデータセットに対する不正解例



図 19: 手法(5), 簡単なデータセットに対する正解例



図 20: 手法(5), 簡単なデータセットに対する不正解例



図 21: 手法(5), 難しいデータセットに対する正解例



図 22: 手法(5), 難しいデータセットに対する不正解例



図 23: 手法(6), 簡単なデータセットに対する正解例



図 24: 手法(6), 簡単なデータセットに対する不正解例



図 25: 手法(6), 難しいデータセットに対する正解例



図 26: 手法(6), 難しいデータセットに対する不正解例



図 27: 手法(7), 簡単なデータセットに対する正解例



図 28: 手法(7), 簡単なデータセットに対する不正解例



図 29: 手法(7), 難しいデータセットに対する正解例



図 30: 手法(7), 難しいデータセットに対する不正解例



図 31: 手法(8), 簡単なデータセットに対する正解例



図 32: 手法(8), 簡単なデータセットに対する不正解例



図 33: 手法(8), 難しいデータセットに対する正解例



図 34: 手法(8), 難しいデータセットに対する不正解例



図 35: 手法(9), 簡単なデータセットに対する正解例



図 36: 手法(9), 簡単なデータセットに対する不正解例

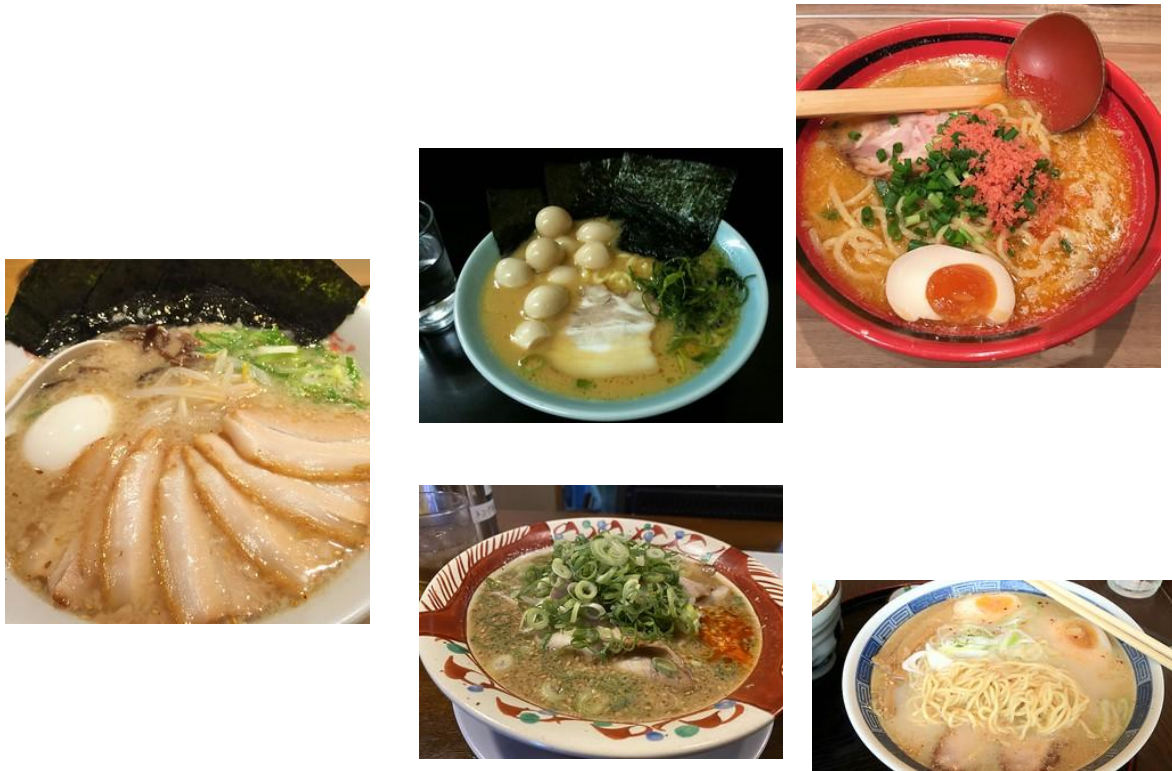


図 37: 手法(9), 難しいデータセットに対する正解例



図 38: 手法(9), 難しいデータセットに対する不正解例



図 39: 手法(10), 簡単なデータセットに対する正解例



図 40: 手法(10), 簡単なデータセットに対する不正解例



図 41: 手法(10), 難しいデータセットに対する正解例



図 42: 手法(10), 難しいデータセットに対する不正解例



図 43: 手法(11),簡単なデータセットに対する正解例



図 44: 手法(11),簡単なデータセットに対する不正解例



図 45: 手法(11), 難しいデータセットに対する正解例



図 46: 手法(11), 難しいデータセットに対する不正解例

次に、課題 2 の結果についてまとめる。まず、評価基準としてはラーメンを撮ろうとしてラーメンを映したものの以外はラーメンとはしなかった。例えばラーメン屋の画像やラーメン屋店内の様子でラーメンが写っていたとしてもラーメン画像とはしなかった。その代わりラーメンの範囲は広めにとり、カレーのような色をしていて見たことないような麺類でもラーメンに似ていると思ったものはラーメンに含めた。寿司についても同様であるが、さすがに海鮮丼は寿司には含めていない。以下の表 2 に結果をまとめた。ただし n は上位何番目まで学習データを取ってきたかを表す。

表 2: 課題 2 の実験結果

n	キーワード	リランキング前の正	リランキング後の正
		解枚数	解枚数
25	ラーメン	34	100
50	ラーメン	34	100
25	寿司	30	65
50	寿司	30	62

以下の図 47 ~ 50 に検索結果の上位の画像を示す。

Rank	Score	Image
1	1.0827	
2	1.0590	
3	1.0539	
4	1.0195	
5	0.9935	

図 47: $n = 25$ のラーメン画像の
リランキング上位 5 枚

Rank	Score	Image
1	1.5709	
2	1.5098	
3	1.4664	
4	1.4610	
5	1.4326	

図 48: $n = 50$ のラーメン画像の
リランキング上位 5 枚

Rank	Score	Image
1	0.7807	
2	0.7052	
3	0.5686	
4	0.5297	
5	0.5275	

図 49: $n = 25$ の寿司画像のランキング上位 5 枚

Rank	Score	Image
1	1.3079	
2	1.1196	
3	0.8575	
4	0.8544	
5	0.7667	

図 50: $n = 50$ の寿司画像のランキング上位 5 枚

5. 考察

まず、課題 1 について考察していく。実験の結果、図 1 や表 1 より、Color Histogram や BoF といったハンドクラフト特徴量を用いた従来手法と比較して、DCNN を用いた手法が、簡単なデータセットと難しいデータセットの双方において高い分類精度を示した。

Color Histogram は画像の色彩分布のみを特徴とするため、簡単なデータセットにおいてはラーメンのスープの茶色や黄色と関係ない画像の色味の違いでおおよその分類が可能であると考えられるが、難しいデータセットにおいては両者とも茶色いスープ、白い麺という類似した色彩構成を持つため、色情報だけでは識別が困難であり、精度が低下したと考えられる。

一方、DCNN は色だけでなく、エッジ、テクスチャ、どんぶりの丸みなどの形状といった階層的な特徴を自動的に学習している。これにより、色彩が酷似している麺類同士の分類においても、麺の太さや縮れ具合、具材の有無といった微細な違いを捉えることができ、高い精度を達成したと考えられる。

図 2 より、DCNN のモデル間比較を行うと、層が比較的浅い AlexNet に比べ、より深い層を持つ VGG16、さらに深い ResNet101 や DenseNet201 の方が高い精度を示す傾向が見られた。ネットワークが深くなることで、より抽象度が高く、複雑な識別境界を持つ特徴表現を獲得できたと考えられる。特に ResNet は残差学習により、DenseNet は特徴マップの再利用により、勾配消失問題を回避しながら深い層での学習を可能にしている。簡単なデータセットでは AlexNet でも既に飽和に近い精度が出ていたが、難しいデータセットのような微細な差異の識別が必要な場面では、深層構造による表現力の差が顕著に表れたと考える。

また、DCNN 特徴量を分類器に入力する際、線形 SVM と非線形 SVM を比較した結果、非線形 SVM の方がわずかに高い精度を示したが、線形 SVM でも十分に高い精度が確認された。これは、DCNN の最終層付近で抽出される特徴量が、すでに高次元空間上でクラスごとに綺麗に分離しやすい状態に変換されていることを示唆していると考えられる。非線形 SVM は計算コストが高い一方、線形 SVM は高速に学習、予測が可能である。DCNN 特徴量を用いる場合、実用的には線形 SVM でも十分な性能が得られることが本実験から確認できた。

実験において誤分類された画像について考える。図 4 より Color Hist は前述のとおり色味が似ているものや背景がしっかり写っているものを誤ってしまう傾向がみられる。Bof については図 8, 図 10 より、ラーメンがアップに撮られすぎていてどんぶりの縁が切れ、エッジとして認識しにくい写真を不正解と判断してしまったり、逆にそばについているカツどんのどんぶりやそばつゆの器をどんぶりの丸いエッジと似ていると判断して誤認してしまう傾向があると考えられる。Alexnet においては、図 12, 図 14 より、丸いものを誤認しているように見える。特にピザの画像は丸と具材の要素を持っており、分類が非常に難しいと考えられる。また、難しい方のデータセットにおいて四角い容器のそばについても誤認していることから麺の要素もしっかり読み取ることができていると考えられる。さらに、densenet において図 28, 41 よりやはり丸い形の中の具材をラーメンと誤認してしまう傾向がみられ、今回実験したネットワーク全てにこれらの画像が含まれている。これらのネットワークでは食パンの画像もピザの画像も丸には立体感がなく、どんぶりの立体感をとらえることができていないのではないかと考えられる。人間の目が特別なものかもしれないが、2次元画像から3次元に復元したときのことを想像

するような計算機構を取り入れることができれば精度はより上がるかもしれないと考えられる。

次に課題2について考察していく。実験の結果、キーワード「ラーメン」「寿司」の双方において、再ランキングを行うことで上位100枚に含まれる正解画像の枚数が大幅に増加した。具体的には、表2のとおりである。これは、Flickrのinteresting, latestの順での検索結果に含まれるノイズをrelevancの上位画像からDCNN特徴を抽出して学習したSVMによって効果的に排除できたことを示していると考えられる。

両者とも精度は向上したが、ラーメンが100%という完璧な精度を達成したのに対し、寿司は60%台に留まった。この差が生じた最大の要因は、Flickrでの収集時に設定したソート順の違いによるデータセットの質と安定性にあると考えられる。ラーメン画像についてはinteresting順で集めた。これは、人気順に表示されていると考えられ、比較的高品質で構図が整った画像が上位に来る設定であると考えられる。これは学習データと画像の性質が近く、特徴量の分布が類似していたため、SVMが容易に決定境界を引くことができ、高い精度を達成できたと考えられる。また、ラーメンの画像は安定していることが多いと考えられる。これは、円形のどんぶり、茶色っぽいスープ、麺の線形構造といった共通の視覚的特徴を持ちやすいという意味である。撮影アングルも斜め上からの構図が多く、統一感がある。そのため、少数の学習データでも決定境界を容易に学習でき、Flickr内の多様なノイズとも明確に分離できたと考えられる。一方で、寿司はlatestで収集したため、ユーザーがアップロードしたばかりの無選別の画像が対象となった。ここには、ピンボケ、極端な暗所撮影、食べかけの皿、あるいは寿司とは無関係なスクリーンショットなどの低品質な画像や外れ値がラーメンよりも多く含まれていると考えられる。学習データとテストデータの質の乖離が大きかったことが、ラーメンに比べて精度が伸び悩んだ一因であると考えられる。さらに、寿司は視覚的なバリエーションが極めて豊富である。握り寿司、軍艦巻き、太巻き、ちらし寿司など形状が多岐にわたり、皿の形状や色も一定ではない。このような不安定性が決定境界を引くことの難易度をさらに高めたと考えられる。ここで、寿司において学習データを $n=50$ に増やした際に精度が微減した現象について考える。学習データとして収集した画像は課題1の様にきれいにノイズをそぎ落とす作業を伴っていない。したがって、イラストやノイズ画像が含まれる確率が高まる。特に寿司のように形状が多様なクラスでは、不適切な画像をポジティブとして学習してしまったことで、ただでさえ難しいlatest画像の分類において、誤判定を助長してしまった可能性があると考えられる。

6. 感想

本実験を通して、ハンドクラフト特徴量 (Color Histogram, BoF) から深層学習 (DCNN) への技術的変遷と、それに伴う認識精度の劇的な向上を身をもって体験することができた。特に、カラーヒストグラムでは全く歯が立たなかったラーメンと蕎麦の分類のような難易度の高いタスクにおいて、DCNNがいとも簡単に高精度を達成した点に、深層学習の表現力の高さを強く感じた。

また、課題2のリランキング実験では、Web画像のノイズが検索精度に与える影響の大きさ

を痛感した。寿司の実験において、学習データの質や検索条件の違いが最終的な精度を左右した結果から、アルゴリズムの性能だけでなく、学習データのクレンジングや選定の重要性を学ぶことができた。

実装面では、画像のダウンロードミスやファイルパスの扱いに苦戦したが、エラーログを読み解きながら修正することで、MATLAB による画像処理プログラミングのスキルも向上したと感じている。今後は、より新しいネットワークアーキテクチャとの比較も行っていきたい。

7. 付録 1

以下に github のリンクを示す。課題 1 で使うコードは `makeLimg.m`, `makeCodebookBoF.m`, `repo1.m` の 3 つである。

<https://github.com/s2311119/image-recognition-report/tree/main>

8. 付録 2

以下に github のリンクを示す。課題 2 で使うコードは `repo2.m` のみである。

<https://github.com/s2311119/image-recognition-report/tree/main>